

08 | 调用第三方：下游的接口不稳定性能又差怎么办？

2023-7-3 邓明



你好，我是大明。今天我们来聊一个跟微服务架构有很强关联的话题：如何保证调用第三方接口的可用性。

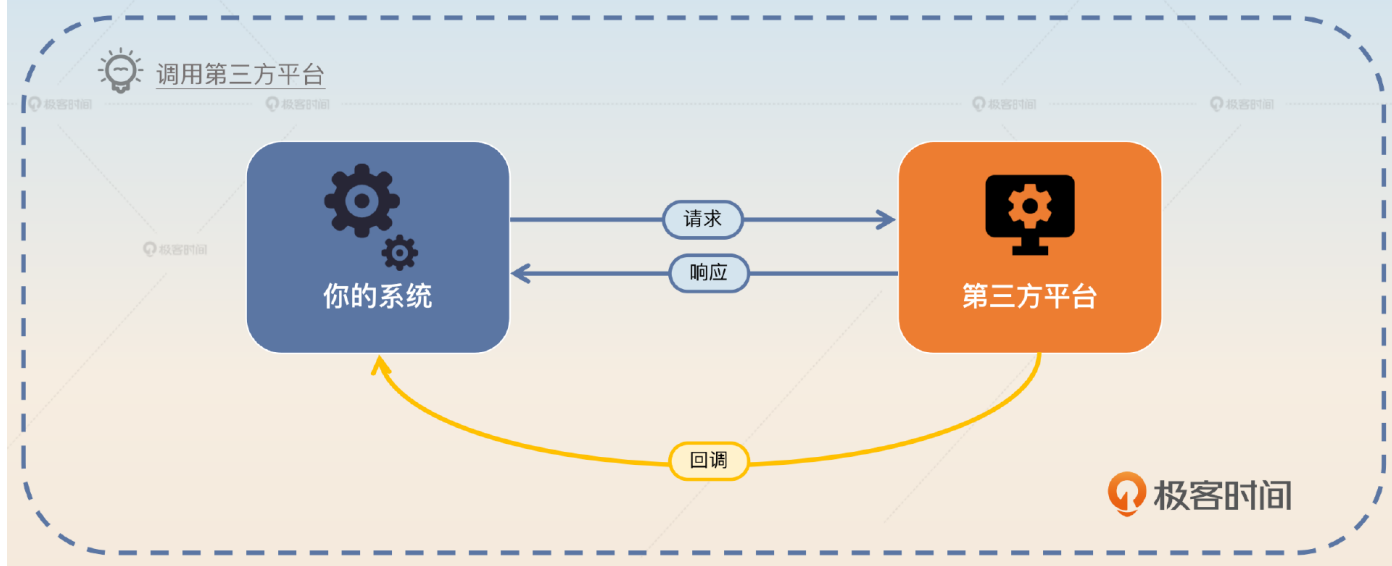
到目前为止，我们可以看到任何一个系统，都难免要跟第三方打交道。

登录注册要跟微信开放平台打交道，接入扫码登录。

金融要跟银行打交道，比如结算。

重要功能发验证码，要跟短信服务商打交道。

人脸识别、身份认证也要跟供应商打交道。



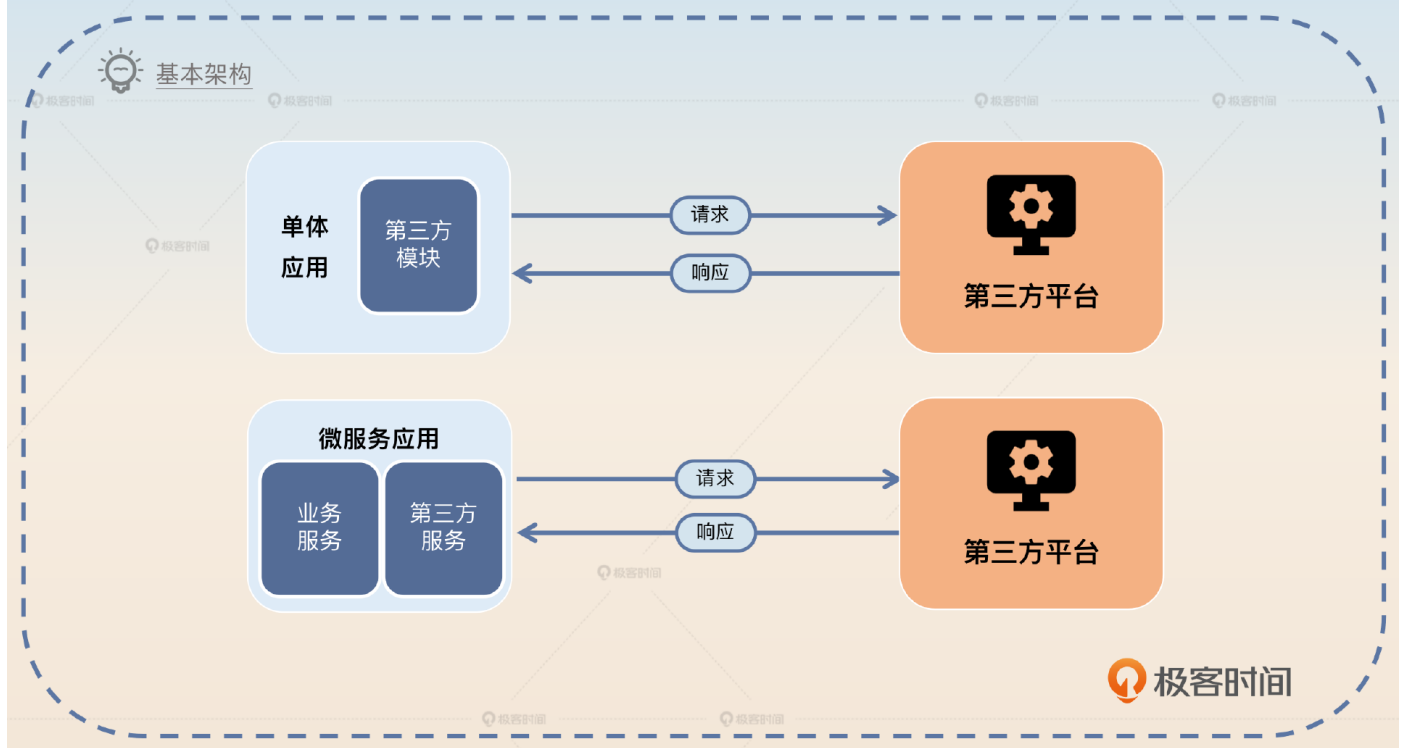
所以早期我就注意到很多人的简历上都写了自己对接过这一类的 API。但是我还注意到大多数人对接这些 API 的时候只是简单实现了功能。换句话说，就是完全没有考虑可用性和容错之类的问题。

实际上，调用第三方接口是一个非常常见的场景，面试官很容易理解，所以在面试的时候你谈到这样的项目，很容易取得共鸣。而且你可以在上面应用非常多的微服务治理措施，和前面学过的内容形成呼应。

所以今天我就来和你深入讨论一下，如果你需要调用一些第三方接口，而你难以推动这个第三方接口的提供者做一些事情的时候，如何保证你的系统高可用。

前置知识

正常来说，和第三方平台打交道的是一个独立的模块还是一个独立的服务，取决于你维护的是一个单体应用还是微服务应用。



对于自己系统内这样一个第三方模块或者第三方服务来说，它要解决的问题也很直观。

提供一个一致性抽象，屏蔽不同第三方平台 API 之间的差异。

提供客户端治理，即提供调用第三方平台 API 的重试、限流等功能。

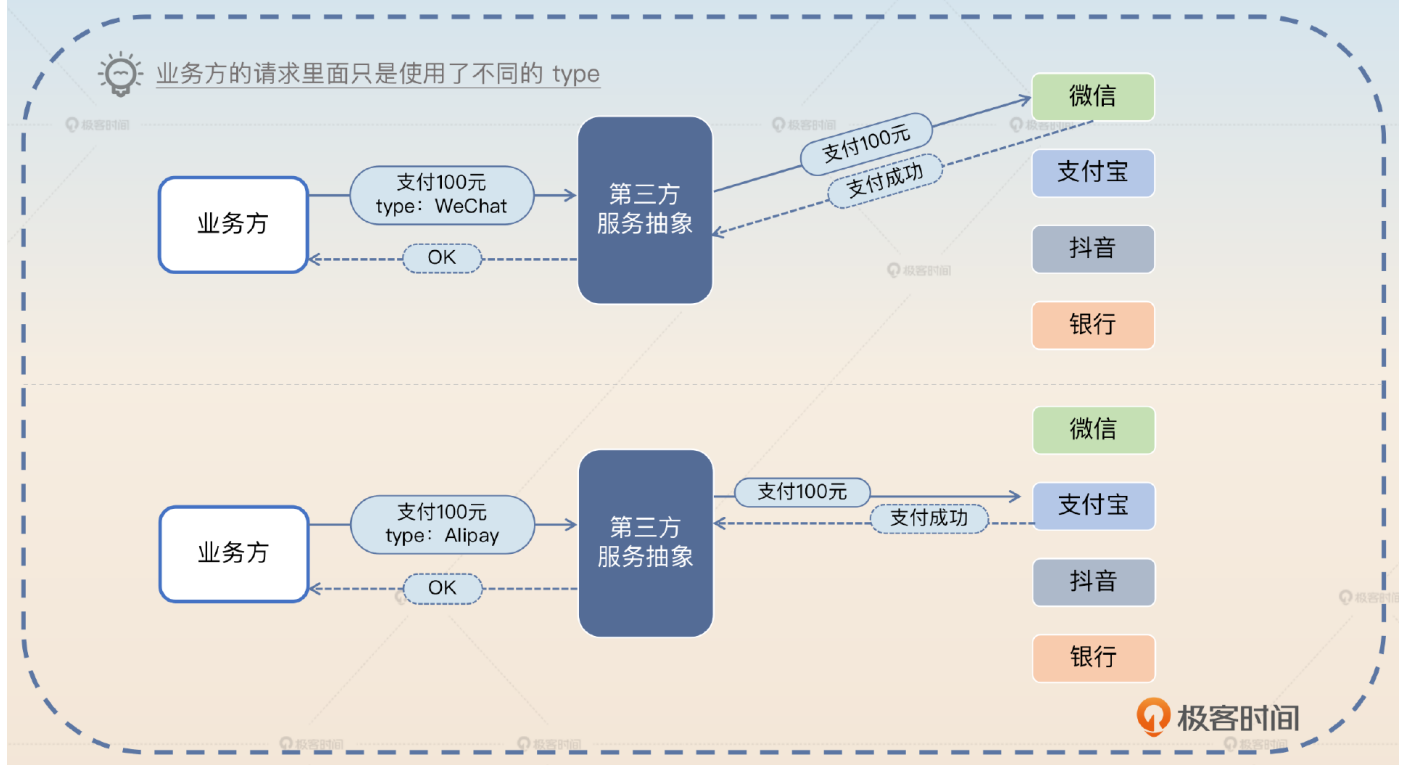
提供可观测性支持。

提供测试支持。

一致性抽象

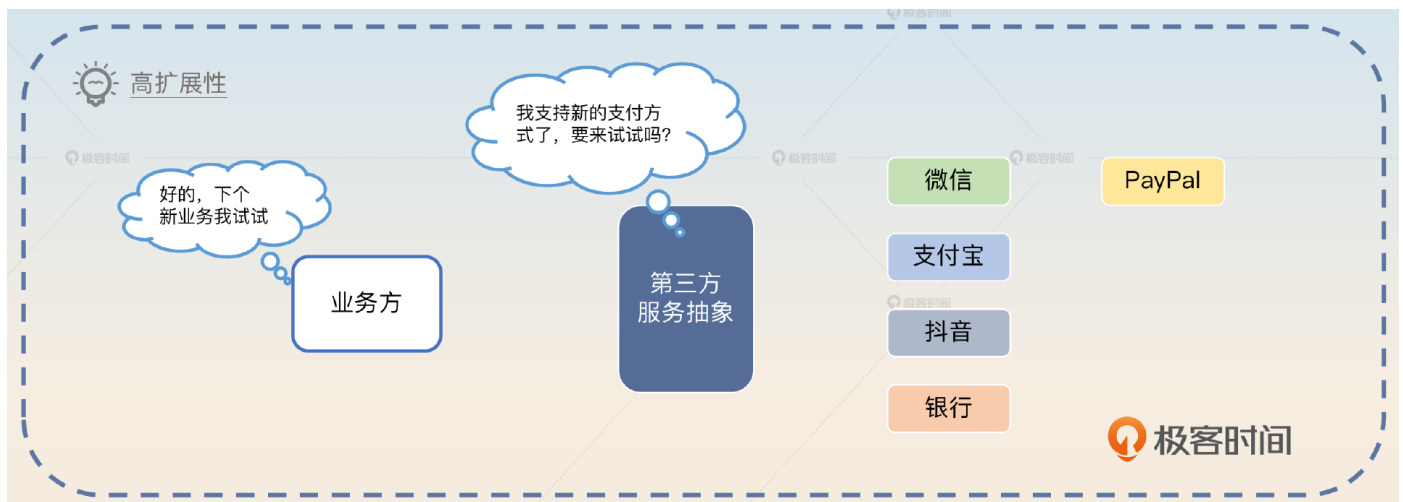
这算是你这个模块或者服务最基本的目标。举个例子，如果你调用的是第三方支付平台，你们公司支持多种接入方式，包括微信支付、支付宝支付。

在这种情况下，业务方只希望调用你的某个接口，然后告诉你支付所需要的基本信息，比如说金额和方式。你这个接口的实现就能根据具体的支付方式发起调用，业务方完全不需要关心其中的任何细节。



这种一致性抽象会统一解决很多细节问题。比如不同的通信协议、不同的加密解密算法、不同的请求和响应格式、不同的身份认证和鉴权机制、不同的回调机制等等。这会带来两个好处。

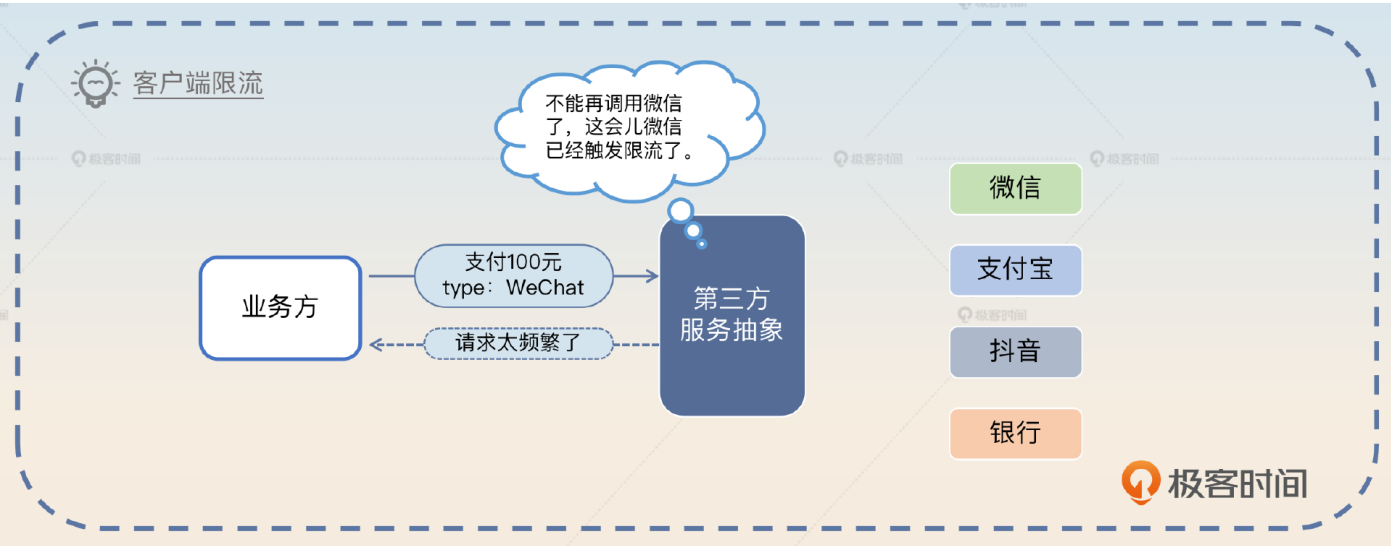
1. **研发效率大幅提高**，对于业务方来说他们不需要了解第三方的任何细节，所以他们接入一个第三方会是一件很简单的事情。
2. **高可扩展性**，你可以通过扩展接口的方式轻松接入新的第三方，而已有的业务完全不会受到影响。



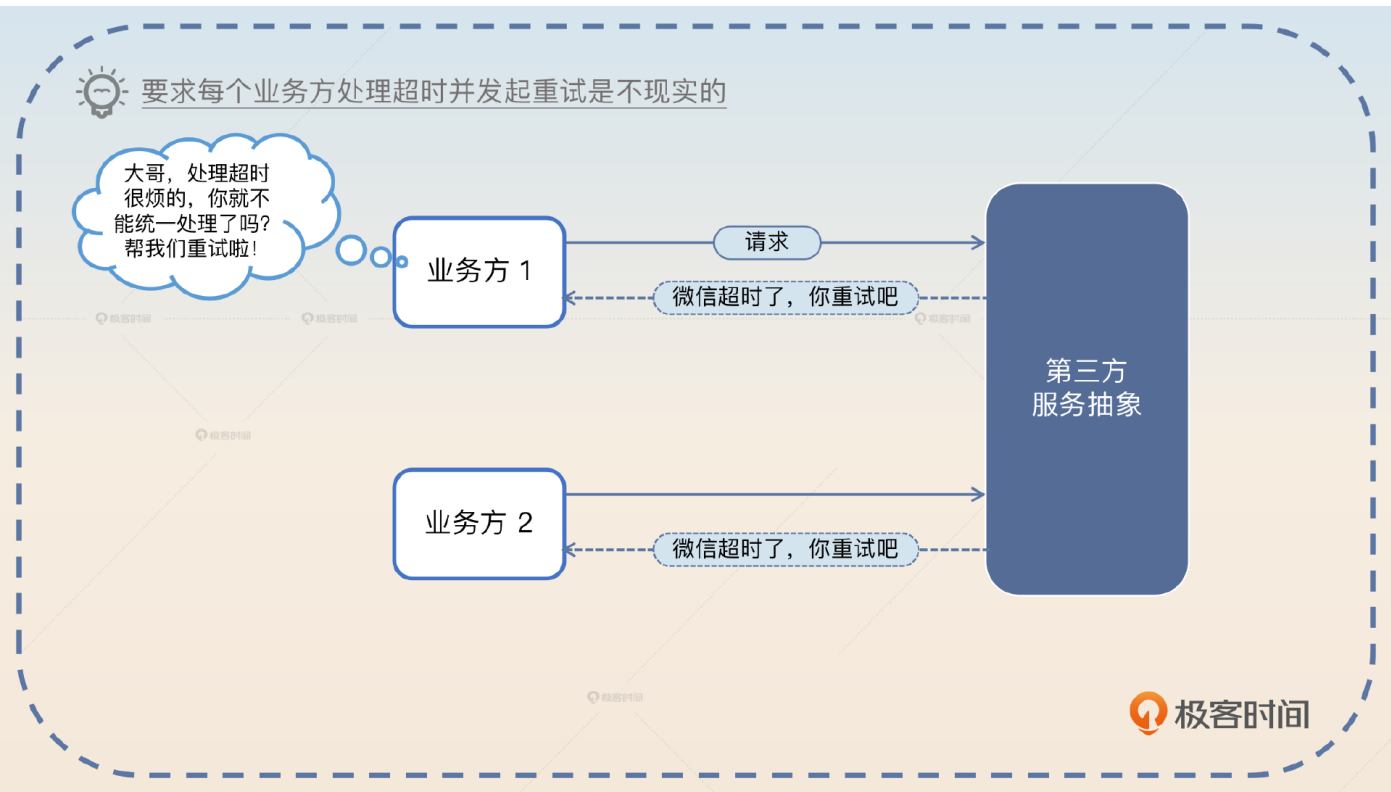
提供了一致性抽象之后，你就可以在这种一致性抽象上做很多事情，比如说客户端治理。

前面我们在讲熔断、降级、限流的时候，实际上都是在服务端或者网关进行的。那么这一次你就需要在客户端进行治理了。一般来说，客户端治理有两个关键措施：**限流**和**重试**。

就拿限流来说，大部分的第三方平台 API 为了保护自己的系统，是不允许你频繁发送请求的。比如说某些银行的接口只允许你一秒钟发送十个请求，多了就会拒绝服务。那么自然地，你其实可以在你发起调用之前就开启限流，这样就可以省去一次必然失败的调用。



另外一个重要的措施是重试。当你调用第三方平台超时的时候，业务方肯定不希望直接返回超时响应，因为他们还要自己处理超时，比如说发起重试等。



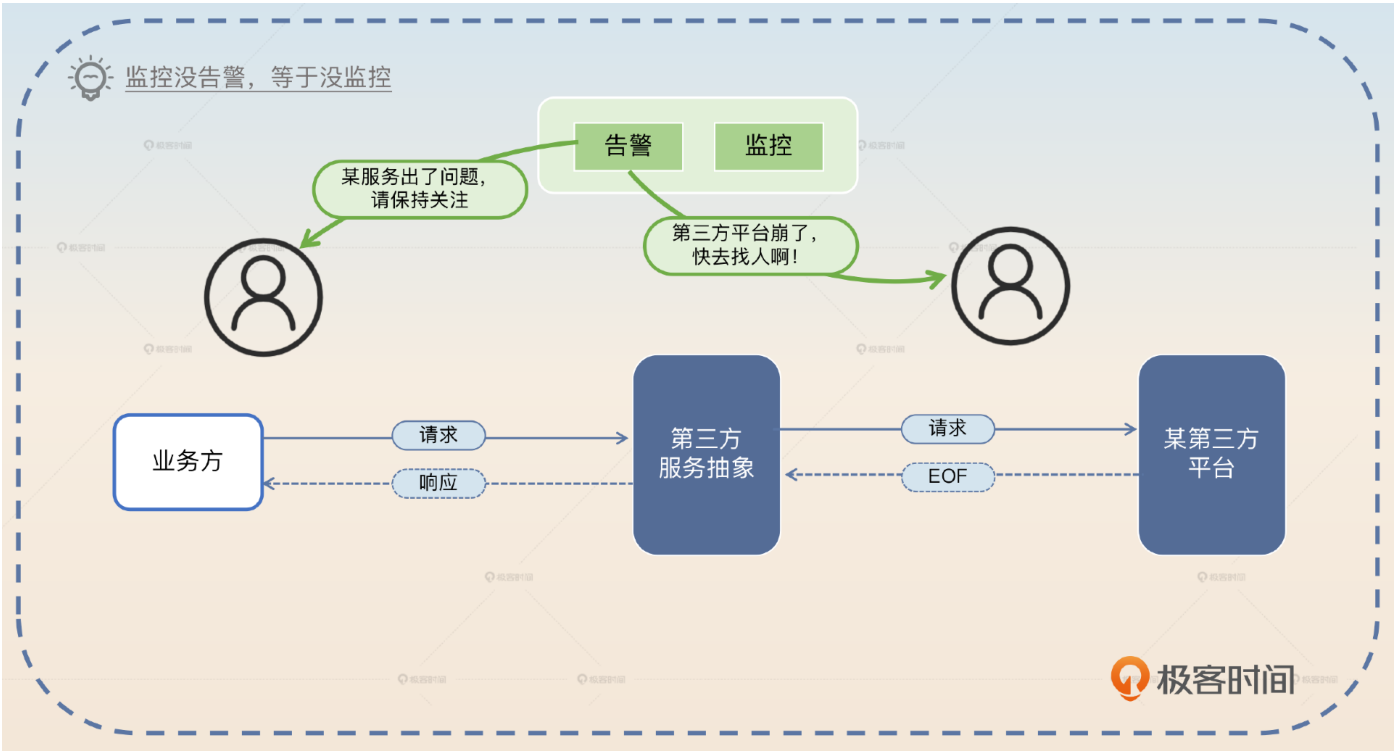
所以你可以提供重试机制，并且可以对业务方保持透明。但你要小心的是，只有当第三方接口是幂等的时候你才能发起重试。

当你完成客户端治理之后，一般是不会出问题的。但万一业务出问题了怎么办？这时候你就需要可观测性支持，告诉你的业务方你的接口稳如泰山，把锅甩出去。

可观测性支持

第三方接口一般都不在你的控制范围内，所以你一定要做好监控，比如说接入 Prometheus 和SkyWalking 等工具。同时，你还要考虑提供便利的查询工具，让你自己和你的业务方都能够快速定位问题。

告警也是必不可少的。这些告警分成两类，一类是给你和你共同维护这个功能的同事使用的，另外一类是给业务方用的。例如，当监控系统发现第三方平台突然不可用了，那么它会发出两个告警，一个是告诉你出事了；另外一个则是通知业务方第三方平台目前不稳定，那么业务方就需要确认对他们业务的影响范围，以及他们是否需要启动一些容错措施。



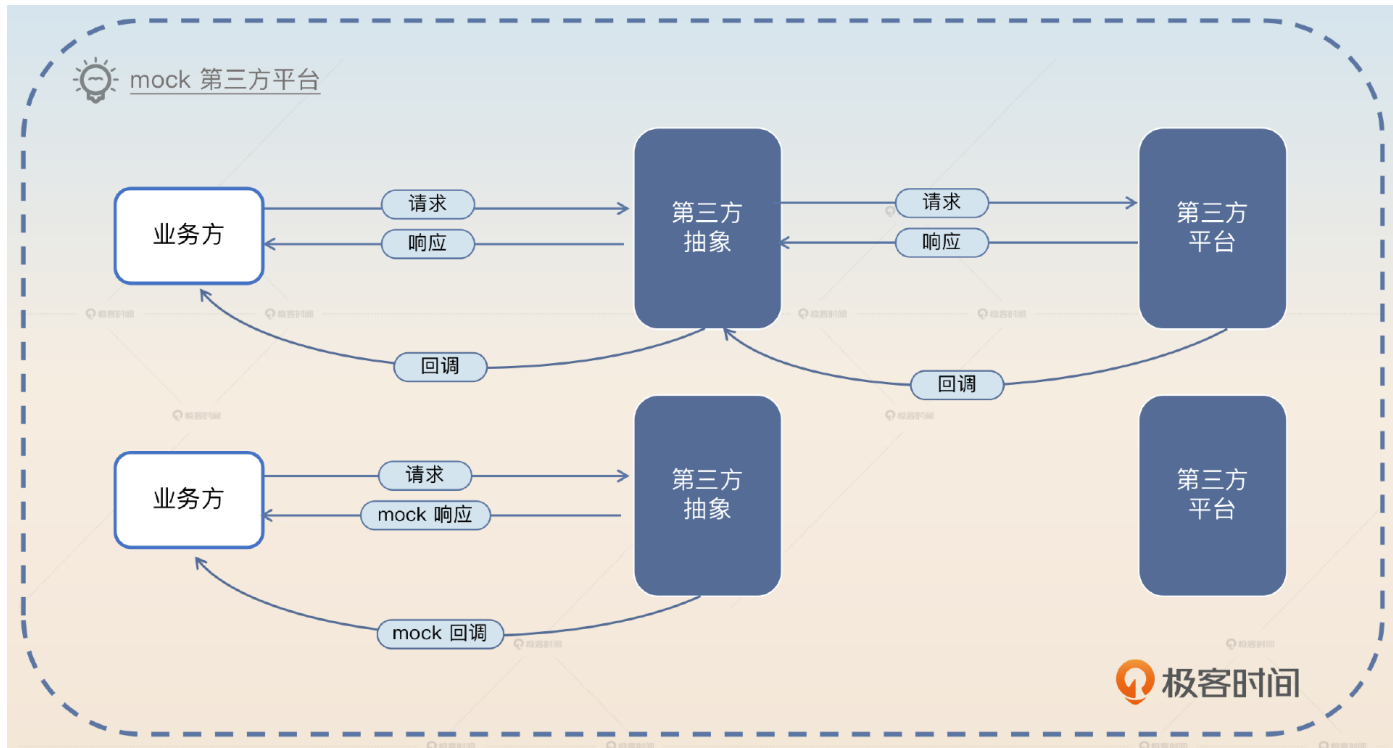
可观测性做得好，定位和解决问题就会变得很简单。但是能不能进一步降低一点出问题的概率呢？

当然是可以的，你把测试支持做好，让你的业务方多测测，省得出问题甩锅给你。

测试支持

测试支持的核心是你要提供**mock服务**。例如正常情况下，业务方调用你的接口，你会真的调用第三方API。但是在测试环境下，你就要考虑返回mock响应。

如果第三方平台还有回调机制，并且你在收到回调之后还要通知业务方，那么你还需要模拟这个回调。比如说微信支付接口后面会回调你的一个接口，告知你支付结果。



使用 mock 服务有很多好处。

没有额外开支。比如说发短信之类的，短信是收费的，那么测试服务如果能避免真的发送短信，多少也能省一点。

不受制于第三方平台。有些第三方平台的认证和鉴权机制非常复杂，在测试环境要发起一次调用几乎不可能，那么只能用 mock 服务。

你可以返回业务方任何预期的响应，包括成功响应、失败响应，甚至于你还能返回模拟第三方平台超时的响应。

如果考虑到压测之类的问题，那么这个 mock 功能就更加必不可少，毕竟第三方是不可能配合你做压测的。

面试准备

如果你的业务需要和第三方平台打交道，那么你需要了解清楚以下信息。

你们是否构建了一个一致性的抽象，屏蔽了不同平台之间的差异？比如说你是和短信服务商打交道，如果你们公司决定换一家短信服务商，那么你需要做哪些事情，你的业务方能否不受影响？

第三方平台有没有治理措施？比如说有没有限流机制，如果有是怎么限流的，你有没有针对这个限流做对应的客户端限流？

你有没有在和第三方打交道的时候引入重试机制，以及重试几次，重试间隔如何，如果重试最终都失败了怎么办？

面试调用第三方这个话题的最佳策略就是将它包装成你整个系统可用性的关键一环。所以在面试官问到可用性相关内容的时候，或者直接问你是怎么和第三方打交道的时候，你就可以和他深入讨论这节课的内容。

调用第三方相关题目

1. 你的系统是怎么保证高可用的？这个问题你要结合前面的熔断、降级、限流和超时控制的内容来一起回答。
2. 你在和 xxx 平台打交道的时候，如果他们的服务出现了问题你的系统会怎样？
3. 在做压测的时候，你怎么解决调用第三方的问题？

基本思路

面试官有些时候不一定会想到要深入考察你和第三方打交道的内容，因为可能他们公司做得就比较差，所以你要考虑主动出击。这种主动出击和前面的熔断、降级、限流、超时控制差

不多。比如说你在自我介绍或者在项目介绍的时候，强调一下你的系统是一个**高可用**的微服务架构。

我的系统对可用性要求非常高，为此我综合使用了熔断、限流、降级、超时控制等措施。并且，我这个系统还有一个特别之处，就是它需要和很多第三方平台打交道。所以要想保证系统的可用性，我就需要保证和第三方打交道是高可用的。

这种话术你已经在前面的内容里见过了。当你将自己的项目说成是高可用的项目的时候，那么面试官肯定会逮着你往死里问高可用，那么你就能将话题全方位展开，并且限定在自己熟悉的战场内。

比如当你们聊到了调用第三方的时候，你可以考虑采用这个话术来介绍你的做法，关键词是**前后对比**。

我在刚接手这个项目的时候，这一块的设计和实现不太行。总体来说可扩展性、可用性、可观测性和可测试性都非常差。为了解决这个问题，全方位提高系统的可扩展性、可用性、可观测性和可测试性，我做了比较大的重构。

1. 我重新设计了接口，提供了一个一致性抽象。（这里你可以补充你设计了哪些接口，然后强调一下效果）重构之后，研发效率提高了 30%，并且接入一个全新的第三方，也能对业务方做到完全没感知。
2. 我引入客户端治理措施，主要是限流和重试，并且针对一些特殊的第三方接口，我还设计了一些特殊的容错方案。
3. 我全方面接入了可观测性平台，包括 Prometheus 和 Skywalking，并且配置了告警。和原来比起来，现在能够做到快速响应故障了。
4. 我还进一步提供了测试工具，可以按照业务方的预期返回响应，比如说成功响应、失败响应以及模拟接口超时。针对压测，我也做了一些改进。



前后对比是常见的面试话术

设计性差

扩展性差

可用性差

性能差

重构前后

设计性好

扩展性好

可用性好

性能好



极客时间

注意，在介绍任何一点的时候都要强调一下你最终取得的效果。这样能够凸显你在改进系统的时候是有计划的、成体系的。

另外这段话里面有一个地方你需要小心，就是研发效率提升30%，这是我举例子说的，而现实中研发效率是很难衡量的。所以你可以换一种说法，用具体例子来说明研发效率的提高。

在重构之前，原本我们公司的 A 组要接入我们的接口，搞了大概一个星期。后面重构之后，B 组接入我们的接口，只用了两天。而且稳定性更好，Bug 更少。

类似地，在告警那里你也可以强调在你完成重构前后的对比。

早期我们调用第三方接口的时候，缺乏监控和告警，以至于只有等用户出现问题联系客服的时候，或者业务方发现我们出现故障报告过来的时候，我们才知道出问题了。后面我们接入了监控和告警之后，在第三方接口出问题的短时间内，就能得到通知，然后快速启动各种容错预案，并且通知业务方和第三方。

最后你要进一步总结和引导。

在任何跟第三方打交道的场景之下，都要考虑好第三方崩溃的时候自己的系统怎么容错。公司或者部门内部的调用出现问题了，还可以推动同事快速修复。但是第三方是推不动的，所以只能是我们调用者考虑容错。

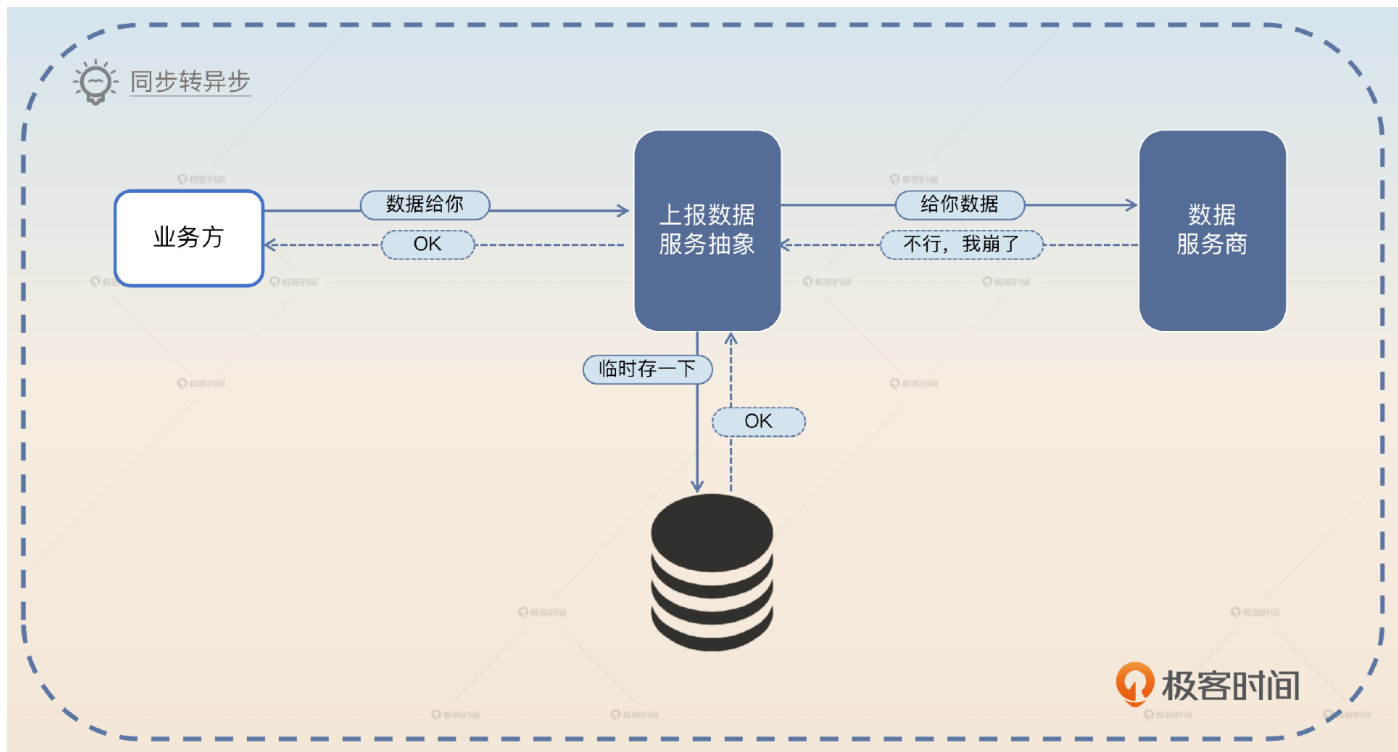
这里我依旧是留了一个话柄，就等着面试官来问怎么容错，这也就是我在亮点方案里面写的前两点，你可以考虑选择其中符合你业务的来回答。

亮点方案

这里我提供三个可以刷亮点的方向，分别是**同步转异步**、**自动替换第三方**和**压测支持**。你可以根据你的需要与面试的情况，选择其中一两个。

同步转异步

在一些不需要立刻拿到响应的场景，如果你发现第三方已经崩溃了，你可以将业务方的请求临时存储起来。等后面第三方恢复了再继续调用第三方处理。这种方案一般用于对时效性要求不高的业务。比如业务方只是要求你上报数据，不要求你立刻成功，那么你就可以采用这种方案。



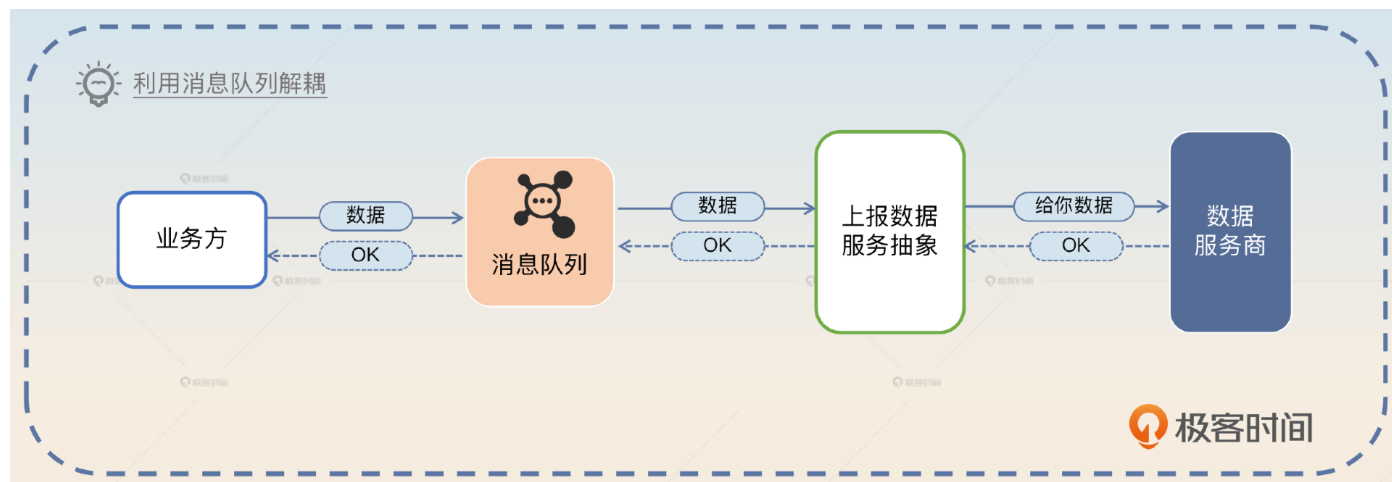
你可以仔细介绍你的容错方案，关键词就是**同步转异步**。

正常来说我们推送数据都是尽可能实时推过去，但是有些时候业务方推过来的数据太多，又或者第三方崩溃，那么我就会临时将数据存起来。后面第三方恢复过来了，再逐步将数据同步过去。这算是比较典型的同步转异步用法。

更进一步，你可以阐述对这个做法的进一步改进，关键词是**解耦**。

我们这种容错机制其实完全可以做成利用消息队列来彻底解耦的形式。在这种解耦的架构下，业务方不再是同步调用一个接口，而是把消息丢到消息队列里

面。然后我们的服务不断消费消息，调用第三方接口处理业务。等处理完毕再将响应通过消息队列通知业务方。



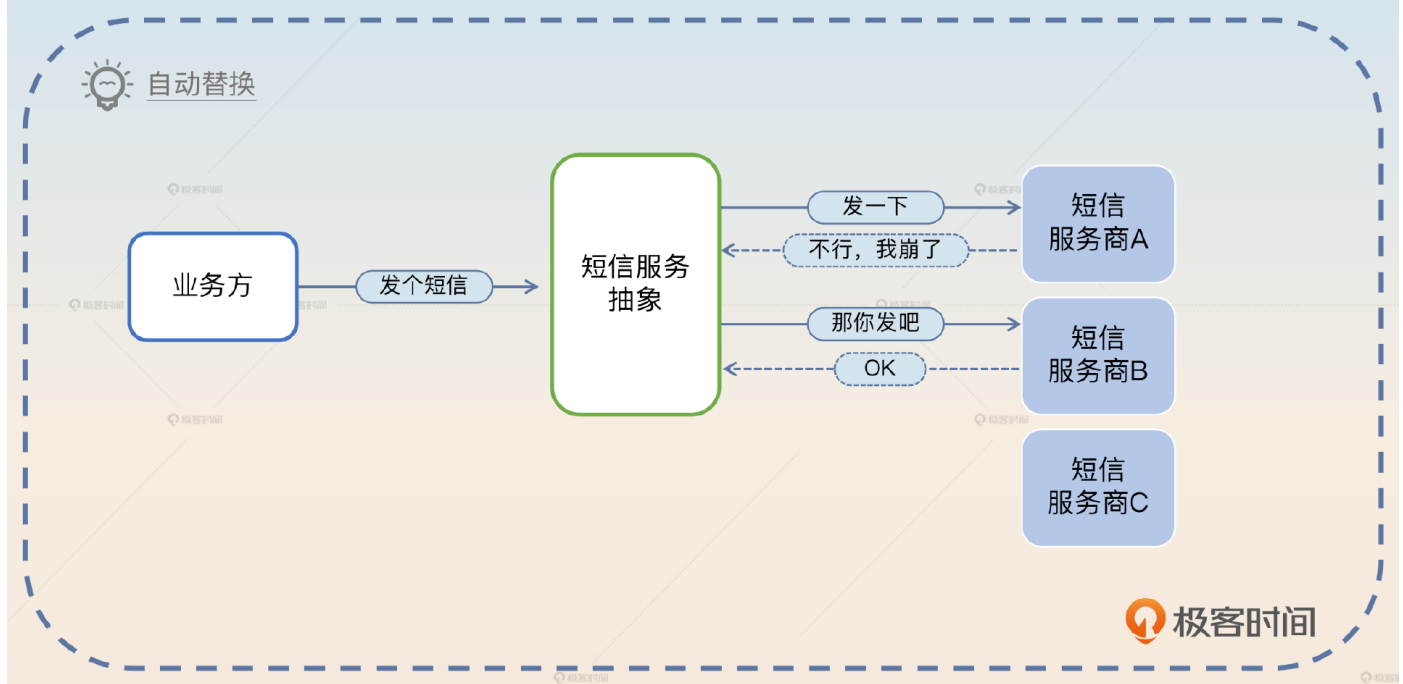
那么这种解耦的方式和直接调用的方式合并在一起，其实就是正常我们系统对接业务方的两个方案。所以你可以再次总结拔高一下。

同步调用与异步解耦两种方式，可以看作是对接不同业务方的通用范式。一般而言但凡能异步解耦的，我绝不搞什么同步调用。

自动替换第三方

这种策略和我在负载均衡里面提到的有些类似，即调用一个第三方的接口失败的时候，你可以考虑换一个第三方。

举例来说，你们公司有 A、B、C 三个短信供应商。现在你在选择使用 A 的时候，发现 A 一直返回失败的响应，或者说响应时间很长，那么你就可以考虑自动切换到 B 上。



但是这种策略是受制于公司的情况的，大多数时候公司是没有这种可以切换的服务供应商的。早期我就听过某司使用的短信服务商服务异常，导致网站在一段时间内都无法发送验证码，引发了严重事故。所以你可以这样介绍你的方案，关键词是**自动替换**。

为了进一步提高可用性，降低因为第三方故障引起事故的概率，我在调用第三方这里引入了自动切换机制。我们本来有多个第三方，相互之间是可以替换的，于是我就做了一个简单的自动切换机制。当我发现第三方接口出现故障的时候，就会切换到一个新的第三方。

这里有一些面试官可能会追问的点。

1. 你怎么知道第三方出问题了？这个问题可以参考我们前面讲过多次的判断服务健康与否的方式，比如说用响应时间、错误率、超时率。那么自然可以将话题引导到熔断、降级、限流那边。
2. 如果全部可用的第三方都崩溃了怎么办？这种问题直接认怂就可以。因为一家出故障是小概率，多家同时出故障那就更是小概率事件了，在这种情况下你除了告警也没有别的办法了。也就是所谓的尽人事，听天命。

压测支持

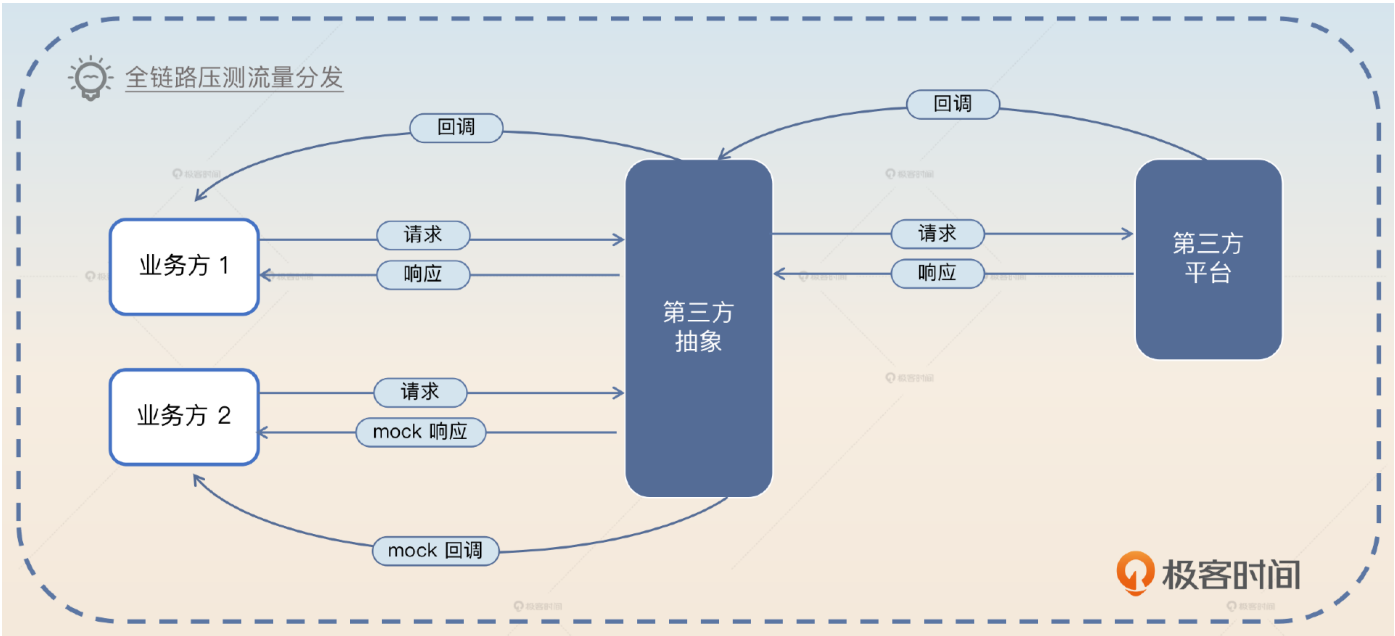
每当你想搞压测的时候，你就会发现，所有的第三方接口都是压测路上的拦路虎。

正常来说，你不能指望第三方会配合你的压测。你可以设想，类似于微信之类的开放平台是不可能配合你搞什么压力测试的。甚至即便你是非常强硬的甲方，你想让乙方配合你做压力测试，也是不现实的。所以你只能考虑通过 mock 来提供压测支持。和正常的测试支持比起来，压测你需要做到三件事。

模拟第三方的响应时间。

模拟触发你的容错机制。如果你采用了同步转异步这种容错机制，那么你需要确保在流量很大的情况下，你确实转异步了；如果你采用的是自动切换第三方，那你也要确保真的如同你设想的那样真的切换了新的第三方。

流量分发。如果是在全链路压测的情况下，压测流量你会分发到 mock 逻辑，而真实业务请求你是真的调用第三方。



那么你在介绍你的测试支持的时候可以强调一下这个特性。

早期为了弄清楚服务的吞吐量和响应时间瓶颈，我搞过一些压测。但这些流量不能真的调用第三方，所以我为了解决压测这个问题，设计了两个东西。

一个是模拟第三方的响应时间。不过这种模拟是比较简单的，就是在代码里面睡眠一段时间，这段时间是第三方接口的平均响应时间加上一个随机偏移量计算得出的。另一个是在并发非常高的情况下，会触发我的容错机制。

而且我这里留好了接口，万一我们公司要做全链路压测了，我这边也可以根据

链路元数据将压测流量转发到 mock 逻辑，而真实业务请求则会发起真实调用。

最后一段我是假设你并没有实际接触过全链路压测。如果你接触过，那么你就可以改成“你已经做到”，而不是“留好了接口”。

面试思路总结

这节课我们讨论了调用第三方平台接口如何保证可用性的问题。当你和第三方平台打交道的时候，要做到这四点：**一致性抽象**、**客户端治理**、**可观测性支持**和**测试支持**。同时在后面我进一步提供了**同步转异步**，**自动替换第三方**和**压测支持**三个亮点方案。

这节课的内容你也可以看作是我们前面讲的熔断、限流、降级和超时控制在一个具体场景下的综合运用。不管你们公司用不用得上这些治理手段，你都要自己去尝试一下，因为面试需要你用上这些治理手段。不然你的整个项目经历平平无奇，那么连面试的机会都难得，更不要说刷出亮点了。

此外，在这节课还有一个点你需要额外注意，就是**强调自己对研发效率的改进**。研发效率是一个比较难量化的东西，所以你在面试的时候要想办法说服面试官相信你确实提高了研发效率。



思考题

最后你来思考2个问题。

你们公司有没有出现什么因为第三方服务不可用引发的故障？后面你们有没有设计什么改进方案？

你的工作经历中有没有什么内容主要是提高同事研发效率的？如果有，你是怎么向面试官介绍这个项目并且让他相信你确实提高了研发效率的？

欢迎你把你的答案分享在评论区，也欢迎你把这节课的内容分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 07 | 超时控制：怎么保证用户一定能在1s内拿到响应？

下一篇 09 | 综合服务治理方案：怎么保证微服务应用的高可用？

精选留言 6